

Analisi del Report – Modulo 2 – C++ Threads

Walkthrough completo del report, sezione per sezione

Partitioned Hash Join with Duplicates

SPM – Università di Pisa – Prof. M. Torquati – A.A. 2025-2026

Walkthrough completo del report, sezione per sezione dall'inizio alla fine, con la motivazione di ogni scelta, la derivazione di ogni numero, la lettura di ogni grafico/tabella, il confronto con baseline/modulo precedente, i *risultati misurati* dove eseguibili senza cluster, e una sezione finale di ulteriori esperimenti. La struttura segue l'ordine del report originale, così puoi presentarlo scorrendo il PDF mentre mostri il documento al prof.

Contents

Come usare questo documento	2
1 I due cambi di baseline (da spiegare prima del report)	2
1.1 FlatCountMap al posto di unordered_map	2
1.2 Fibonacci al posto di $k \bmod P$	3
2 Parallelization Strategy (sezione 1 del report)	3
2.1 Thread Team and Barrier Synchronization	3
2.2 Adaptive thread count	3
2.3 Histogram, Scatter, Join Local	3
3 Correctness Verification (sezione 2 del report)	3
4 Performance Results (sezione 3 del report)	4
4.1 Strong Scaling	4
4.2 Weak Scaling e Phase Breakdown	4
5 Discussion (sezione 4 del report)	4
5.1 Strong Scaling and Amdahl's Law	4
5.2 Weak Scaling and WSE / Effetto di P	4
5.3 Phase-Level Analysis	5
5.4 Partition Count Sensitivity	5
5.5 Duplicate Density	5
6 Ulteriori esperimenti da fare	5

Come usare questo documento

Segue l'ordine del report (Parallelization Strategy → Correctness → Performance Results → Discussion). Prima però due note che il report dà per scontate ma il prof chiederà: i **due cambi di baseline** rispetto al codice del docente (Fibonacci al posto di $k \bmod P$; FlatCountMap al posto di `unordered_map`). I box *Risultato misurato* sono esperimenti che ho eseguito davvero.

1 I due cambi di baseline (da spiegare prima del report)

1.1 FlatCountMap al posto di `unordered_map`

```
struct Slot{ uint64_t key=~0ULL; uint32_t cnt=0; uint32_t _p=0; }; // 16 B esatti, 4/
// cache-line
FlatCountMap(size_t r){ n=1; while(n<r*2) n<=&1; slots.assign(n,{}); mask=n-1; } //
// load factor <50%
// slot_of(k)=k&mask (IDENTITY); increment/count con linear probing
```

Motivazione della scelta

`unordered_map` è una lista concatenata di nodi su heap: ogni insert una `malloc`, ogni lookup pointer-chasing (cache miss). `FlatCountMap` è un solo `vector` contiguo, linear probing: zero allocazioni per elemento, Slot 16 B $\Rightarrow$ 4/cache-line, capacità $\geq 2|R_p| \Rightarrow$ load factor $< 50\%$. Hash di slot *identity* perché le chiavi sono già uniformi e riusare Fibonacci correlerebbe partizione e slot.

Da dove viene il numero (derivazione)

Load factor $< 50\% \Rightarrow$ sonde attese $\approx \frac{1}{2}(1 + \frac{1}{1-\alpha}) = 1.5$ per $\alpha = 0.5$, indipendenti dalla taglia.

Risultato misurato (sandbox locale, non cluster)

Ho riprodotto `FlatCountMap` vs `unordered_map<uint64,uint32>` (build 5M, probe 10M, output *identico*):

caso	FlatCountMap	unordered_map	rapporto
near-unique (max_key 5M)	0.43 s	0.89 s	FlatCountMap 2.0× più veloce
molti duplicati (max_key 100k)	0.11 s	0.06 s	unordered_map 1.9× più veloce

Lettura onesta (e proprio per questo forte all'orale): nel regime che conta – molte chiavi distinte, tabella grande, cache-unfriendly – `FlatCountMap` vince nettamente (2×), che è il punto del cambio di baseline. *Ma* con pochissime chiavi distinte `FlatCountMap` perde, perché si dimensiona su $|R|$ (numero di *record*) e non sul numero di chiavi *distinte*: alloca una tabella enorme e sparsa che fa thrashing di cache, mentre `unordered_map` cresce solo fino alle chiavi distinte. È un trade-off reale e dichiarabile – e suggerisce un'ottimizzazione (dimensionare su una stima delle distinte). I valori assoluti sono sandbox, non cluster; conta il rapporto.

1.2 Fibonacci al posto di $k \bmod P$

Motivazione della scelta

$k \bmod P$ (P pot. di due) guarda solo i $\log_2 P$ bit bassi: su chiavi strutturate concentra su poche partizioni $\Rightarrow$ il join ciclico produce uno straggler. Fibonacci prende i bit alti del prodotto $\Rightarrow$ partizioni quasi uguali. I due cambi si tengono: partizioni bilanciate sono ciò che rende ottimo (e gratis) il ciclico. (La verifica quantitativa del bilanciamento è nel documento M1: Fibonacci 1.00 anche su chiavi strutturate, $k \bmod P$ collassa a 256.)

2 Parallelization Strategy (sezione 1 del report)

Il report descrive la pipeline a 5 fasi (Histogram, Prefix-Sum, Scatter, Join Local, Accumulation) e la scelta di un *team persistente* a barriere.

2.1 Thread Team and Barrier Synchronization

Motivazione della scelta

La pipeline è bulk-synchronous: ogni fase dipende dall'intera precedente $\Rightarrow$ serve comunque una barriera completa a ogni confine. Un thread pool aggiungerebbe overhead (submission, polling, wakeup) senza sovrapporre nulla; spawn/join per fase gonfierebbe f di $O(k)$ cicli. La *completion function* della `std::barrier` (`noexcept`, niente alloc) gira su un thread tra le fasi: merge istogrammi $O(Pk)$, prefix-sum $O(P)$, calcolo offset.

2.2 Adaptive thread count

Da dove viene il numero (derivazione)

$k = \min(k_{\max}, \lfloor \min(N_R, N_S)/T_{\min} \rfloor)$, $T_{\min} = 8192$ record/thread. Sotto soglia, barriera e traffico inter-thread superano il guadagno. Nei benchmark ($N_R \geq 10^6$) non si attiva mai.

2.3 Histogram, Scatter, Join Local

Il report: istogrammi privati per-thread (zero contesa), merge nella completion; scatter lock-free; join con `FlatCountMap` e assegnazione ciclica.

Da dove viene il numero (derivazione)

Scatter lock-free. Offset per thread t e partizione p : $\text{off}[t][p] = \text{global_begin}[p] + \sum_{j < t} \text{local_hist}[j][p]$. Intervalli disgiunti predeterminati $\Rightarrow$ nessun lock/atomico a runtime. **Attenzione:** lock-free sulle scritture, ma la barriera c'è (rende pronti gli offset). **Ciclico vs LPT:** con partizioni uniformi il round-robin bilancia come LPT a costo zero (LPT richiederebbe un sort $O(P \log P)$ inutile). **Padding alignas(64):** accumulatori su cache line distinte, scritti una volta sola $\Rightarrow$ niente false sharing.

3 Correctness Verification (sezione 2 del report)

Il report: per input piccoli (≤ 500) confronto automatico con un join naive $O(N^2)$ su `join_count` e due checksum (moltiplicatori indipendenti); per input grandi, output seq $\equiv$ par su tutti i k . Verifica fuori dalla regione misurata.

Come difenderlo all'orale

I checksum sono *ordine-indipendenti* (somme modulo 2^{64} di `splitmix64(key)·m`): invarianti rispetto all'ordine di completamento dei thread, quindi seq e par devono dare lo stesso valore – è ciò che rende la verifica valida in parallelo.

4 Performance Results (sezione 3 del report)

Setup: Xeon E5-2640 v2, 2×8 core (16 fisici/32 HT), 2 NUMA, L3 20 MB; $N_S = 2N_R$, $P = 128$, best of 5.

4.1 Strong Scaling

p	$S(10M)$	$E(10M)$	$S(20M)$	$E(20M)$
2	1.40	70%	1.45	72%
8	4.46	56%	4.49	56%
16	7.49	47%	7.59	47%
20	7.41	37%	7.33	37%
32	8.64	27%	9.64	30%

Table 1: $P = 128$. Shaded nel report: $p > 16$ regime HT. Dip a $p = 20$.

4.2 Weak Scaling e Phase Breakdown

Weak ($NR = 1M \times p$): WSE da 0.74 ($p=2$) a 0.33 ($p=32$). Breakdown: scatter $\approx 53\%$ (409 ms), join 309 ms, histogram 51 ms; scatter scala bene anche in HT ($9.7 \rightarrow 13.7\times$), join plateau, histogram $2-3\times$.

5 Discussion (sezione 4 del report)

5.1 Strong Scaling and Amdahl's Law

Il report fitta Amdahl ottenendo $f \approx 0.078$, $S_\infty \approx 12.9\times$, e spiega il divario col picco misurato ($8.64\times$) con la saturazione di banda.

Da dove viene il numero (derivazione)

Come si calcola f (Karp–Flatt). Da $S(p) = 1/(f + (1-f)/p)$: $f = \frac{1/S(p) - 1/p}{1 - 1/p}$.
 $S(16) = 7.49 \Rightarrow f = 0.0758$; $S(32) = 8.64 \Rightarrow f = 0.0872$. Il fit su tutti i punti dà ≈ 0.078 ; $S_\infty = 1/f \approx 12.9$. **Picco $< S_\infty$ non è contraddizione:** Amdahl assume risorse infinite non condivise; il vincolo reale è la banda DRAM condivisa. Firma: f cresce con p ($0.076 \rightarrow 0.087$) – overhead crescente (banda), non frazione seriale costante.

5.2 Weak Scaling and WSE / Effetto di P

Il report attribuisce il calo della WSE allo scatter (scritture casuali, banda condivisa), e mostra che a $P = 512$ il quadro migliora.

Da dove viene il numero (derivazione)

Perché $P = 512$ è l'ottimo. A $P = 128$ ogni FlatCountMap ≈ 4 MB, aggregato (20 thread) 80 MB $>$ L3 20 MB $\Rightarrow$ DRAM-bound. A $P = 512$ tabella ≈ 1 MB, insieme in L3:

$f \rightarrow 0.058$, $S_\infty \rightarrow 17\times$, speedup a $p = 32$ da 8.6 a 11.4 $\times$. La WSE a $p = 32$ migliora da 0.33 a 0.45.

5.3 Phase-Level Analysis

Il report spiega i tre regimi: histogram bus-bound (un incremento per record da 8 B), scatter memory-bound che sfrutta il memory-level parallelism dell'HT, join compute-bound che va in plateau perché le coppie HT condividono le porte ALU.

5.4 Partition Count Sensitivity

Da dove viene il numero (derivazione)

Il bump non-monotono a $P = 32$. A $P=16 < p=20$ il ciclico lascia 4 thread inattivi; a $P=32$ tutti attivi ma 12 thread processano due partizioni con due malloc da 16 MB – stesso carico massimo di $P=16$ ma con un ciclo malloc/free extra senza beneficio di cache. Da $P=64$ le tabelle scendono sotto 8 MB e iniziano a stare in L3, ripristinando la monotonia. Minimo in $P \in [512, 1024]$ (≈ 86 ms).

5.5 Duplicate Density

Da dove viene il numero (derivazione)

La densità nel join. `max_key` = chiavi distinte; piccolo $\Rightarrow$ molti duplicati. Cambia (a) quante coppie produci (`match` $\times$ molteplicità) e (b) quanto sono popolate le tabelle. Misure del report: picco a `max_key`= 10K (7.5 $\times$); cala a 100 (6.4 $\times$: 100 distinte per 128 partizioni $\Rightarrow$ sbilanciamento); 6.9 $\times$ a 10M (near-unique, tabelle DRAM-bound a $P=128$). Il mio microbench (box sopra) conferma la stessa fisica: con pochissime distinte la tabella sovradimensionata soffre.

Come difenderlo all'orale

$S(1) > 1$ (**1.03–1.04**): **non super-linear**. Seq e par sono eseguibili distinti; minimi best-of-5 divergono $\leq 4\%$, varianza intra-run $\leq 3\%$. Offset di misura, dichiarato.

6 Ulteriori esperimenti da fare

Esperimento di supporto

1. **[FATTO] FlatCountMap vs unordered_map**
Come: microbench build+probe
Misura: tempo e correttezza
Atteso: FlatCountMap 2 $\times$ su near-unique, perde su molti duplicati
Dimostra: il guadagno del cambio di struttura e il suo limite (sovradimensionamento) – risultati nel box teal.
2. **Eliminare il dip NUMA col pinning**
Come: `OMP_PROC_BIND=close` + first-touch parallelo
Misura: speedup a $p = 16-22$
Atteso: il dip a $p = 20$ si attenua
Dimostra: che il dip e' NUMA e che M3 lo risolve cosi' (richiede cluster).
3. **Provare il dip remoto**

Come: `numastat -p, perf -e node-load-misses` a $p = 20$ vs 16

Misura: accessi remoti

Atteso: crescita a $p = 20$

Dimostra: la prova diretta dell'ipotesi NUMA.

4. **FlatCountMap dimensionata sulle distinte**

Come: stima le chiavi distinte (es. con l'istogramma) e dimensiona la tabella su quella

Misura: tempo su input ad alta densita' di duplicati

Atteso: recupera il caso in cui oggi perde

Dimostra: che il limite misurato nel box teal e' fixabile.

5. **ThreadSanitizer**

Come: `make hashjoin_par_tsan`

Misura: report race

Atteso: nessuna race

Dimostra: che lo scatter lock-free e la barriera sono corretti.

Collegamento alla teoria

`std::thread/std::barrier`, RAII (L12–L13); BSP; cache, false sharing, MESI; NUMA (L18); Amdahl + **Karp–Flatt**, strong vs weak (L10).